

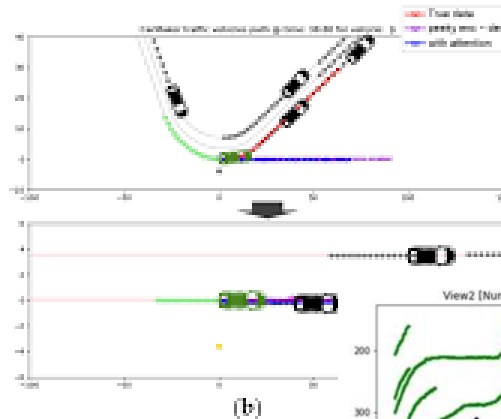


TrajRoute: Rethinking Routing with a Simple Trajectory-Based Approach – Forget the Maps and Traffic!

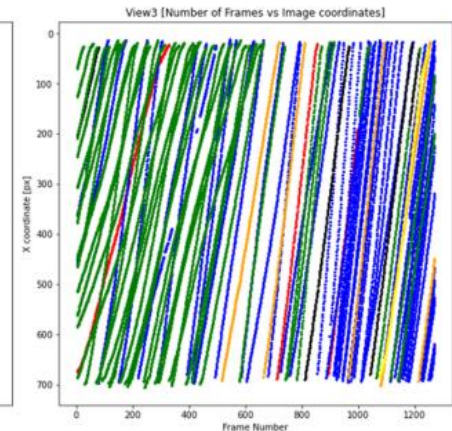
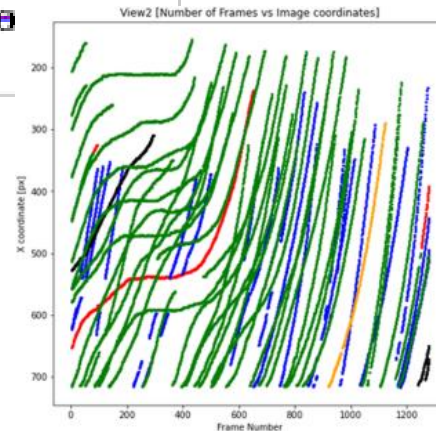
*Maria Despoina Siampou, Chrysovalantis Anastasiou, John Krumm, **Cyrus Shahabi***

University of Southern California (USC)

Reminder: Where does the traffic data currently come from?



Loop Detector



Trajectory Data for Navigation



Massive data availability: GPS-enabled devices generate large-scale vehicle movement data



Industry-scale collection: Companies like Tesla and Google continuously collect real-time trajectories



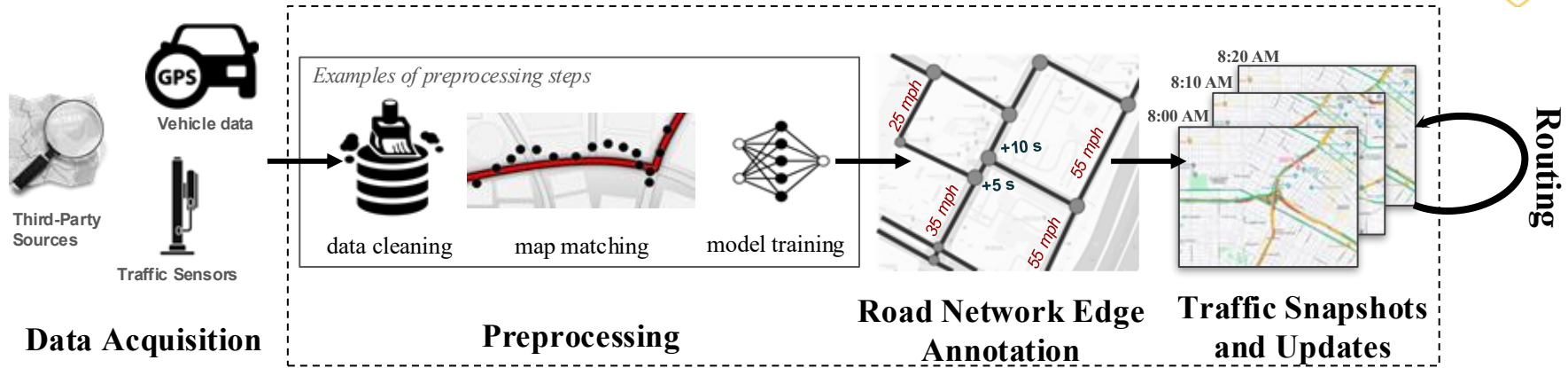
Empirical travel estimates: Real-world GPS traces inform travel times between road segments



Dynamic map refinement: Travel time estimates are refined over time as new trajectory data becomes available.



Typical Pipeline of Routing Services

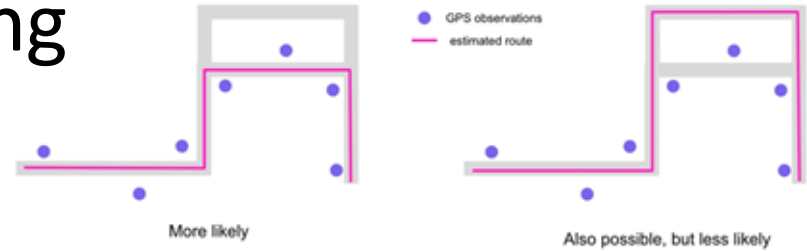


- Large scale, **up-to date** **GPS data** are *continuously collected*
- Several *cost-intensive* preprocessing steps to extract **time dependent “features”**
 - *e.g. Map matching*
- Road network edges are *dynamically updated* (e.g. every 5 minutes) and **new traffic snapshots** are created

Map Matching

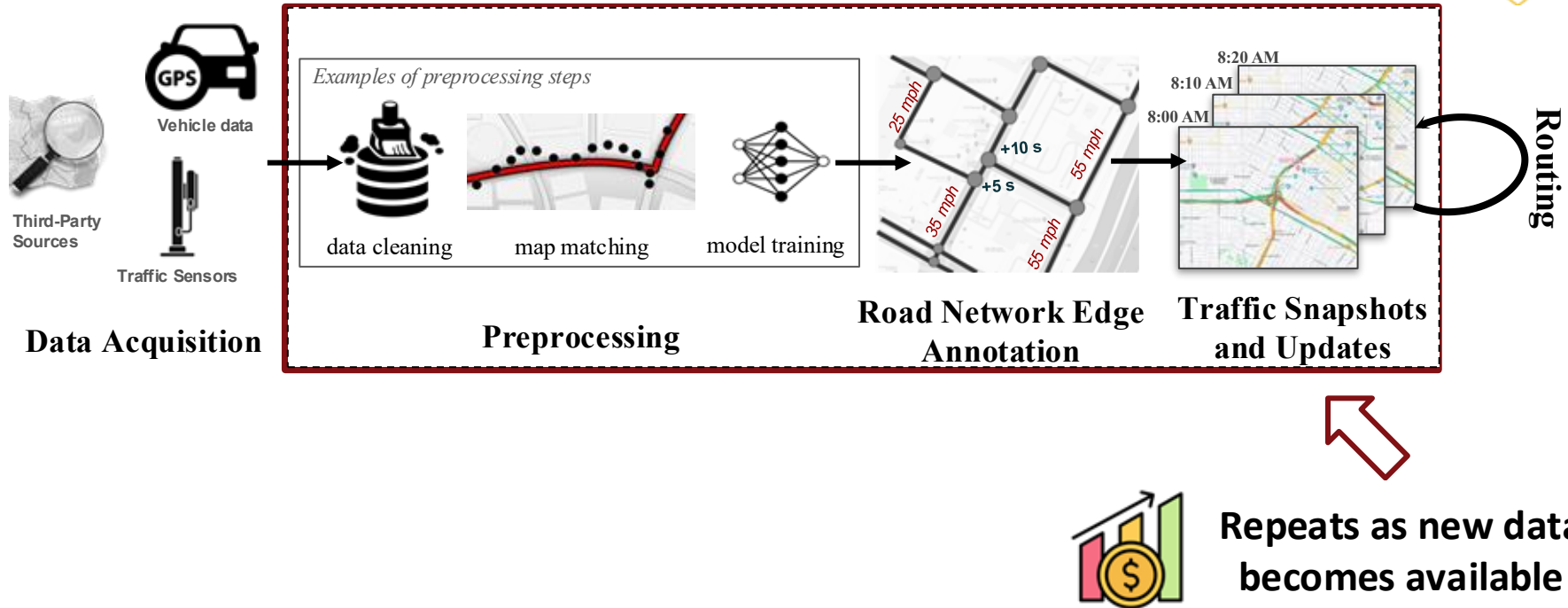


Example of a driver trajectory



- Lyft and Uber use map matching to:
 - To compute the distance travelled by a driver to calculate the fare
 - Dispatch decisions and to display the drivers' cars on the rider app
 - Detect reckless driving
- Why map matching for Origin-Destination Queries?
 - Map past trajectories to road segments
 - Utilize the features of those segments
- Approaches for Map Matching
 - Hidden Markov Model: *Newson & Krumm @ SIGSPATIAL '09* [1]
 - DiDi's IJCAI-19 Tutorial [2]
 - Map Matching @ Uber [3]

Typical Pipeline of Routing Services

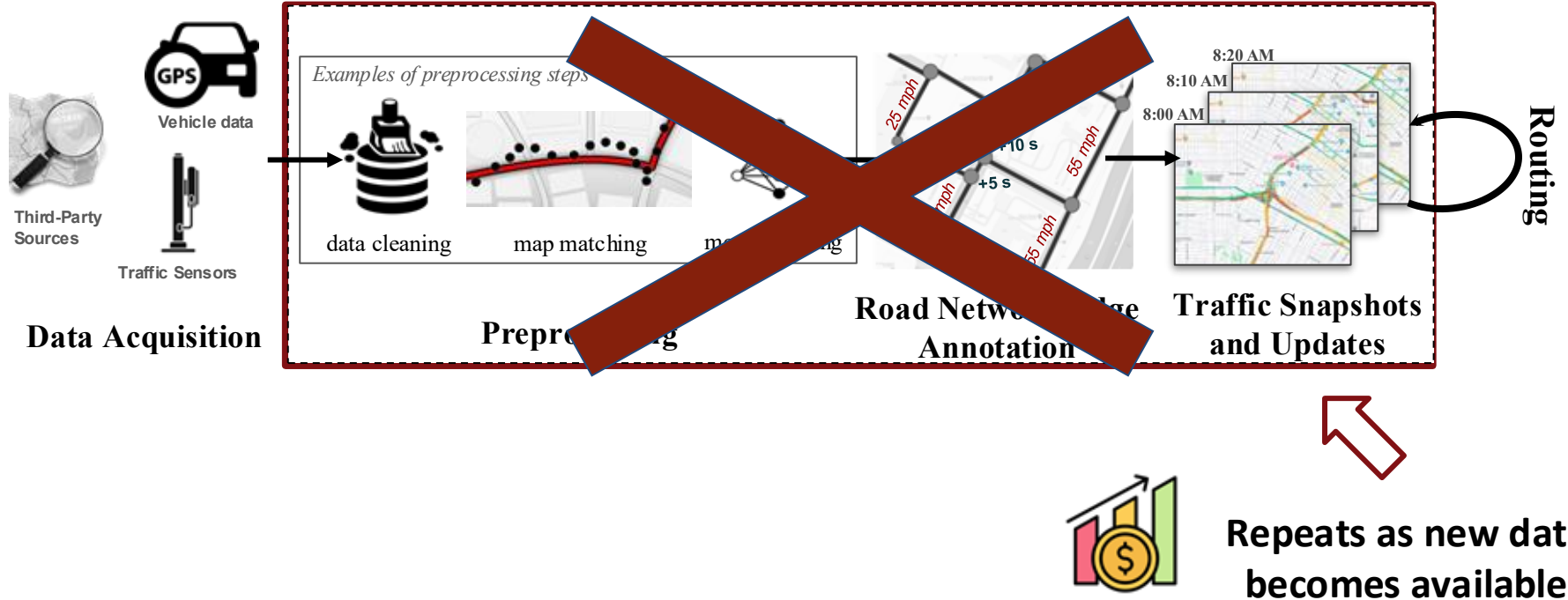


Outline

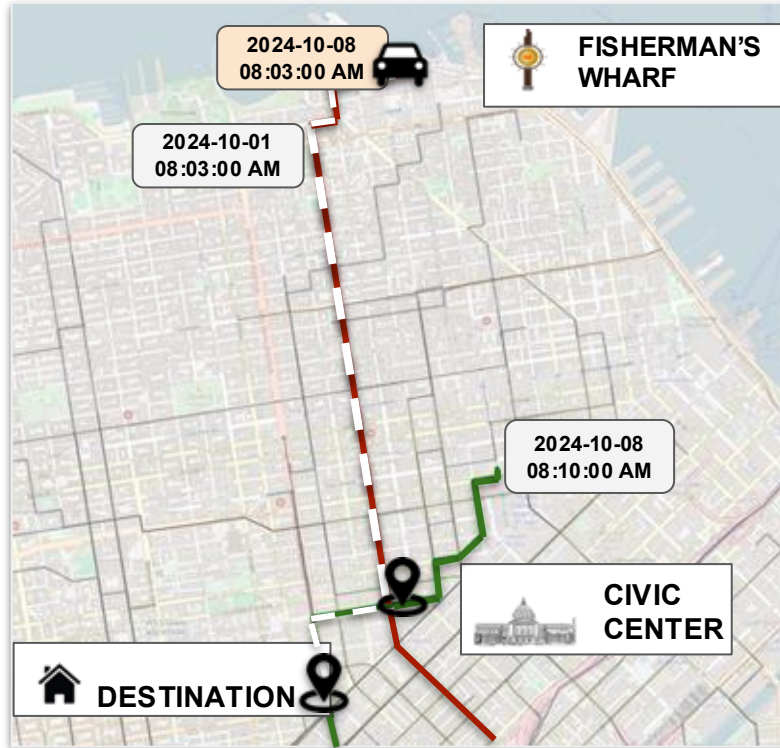


- Motivation
- TrajRoute
 - Key Idea
 - Neighbor Identification
 - Fallback Mechanism
 - Origin-Destination Queries
- Experiments
- Conclusion

Typical Pipeline of Routing Services



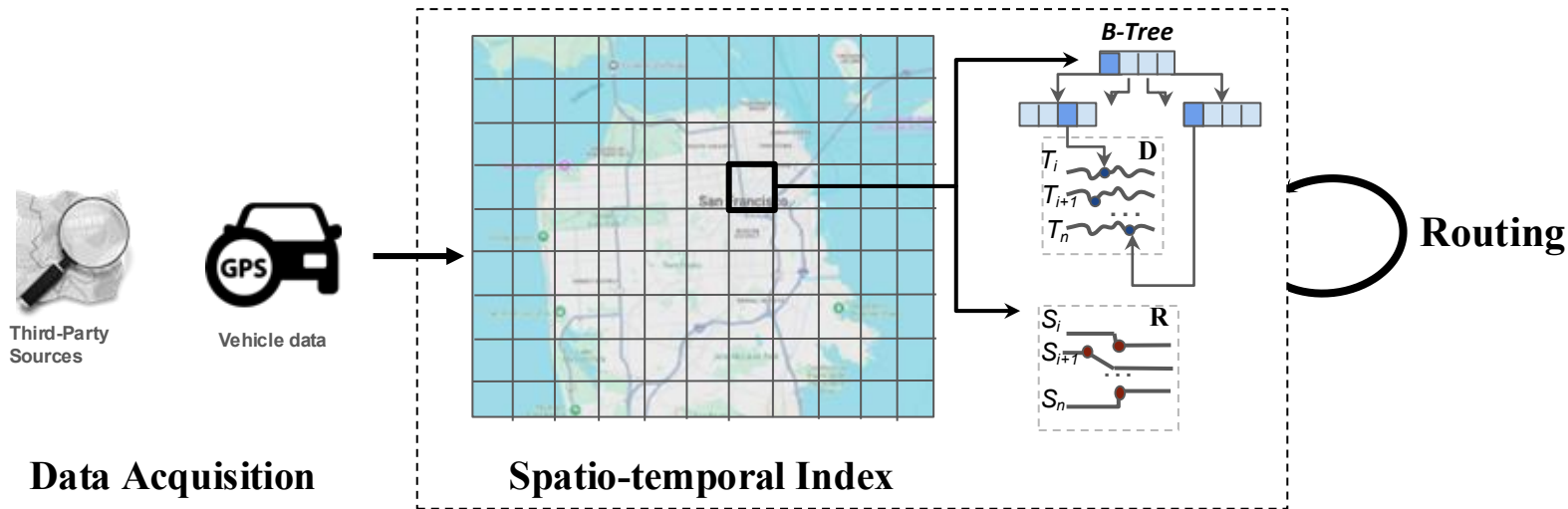
TrajRoute: *Key Idea*



Origin: Fisherman's Wharf
DEST: Home
Time: 08:03:00 AM



Index Structure

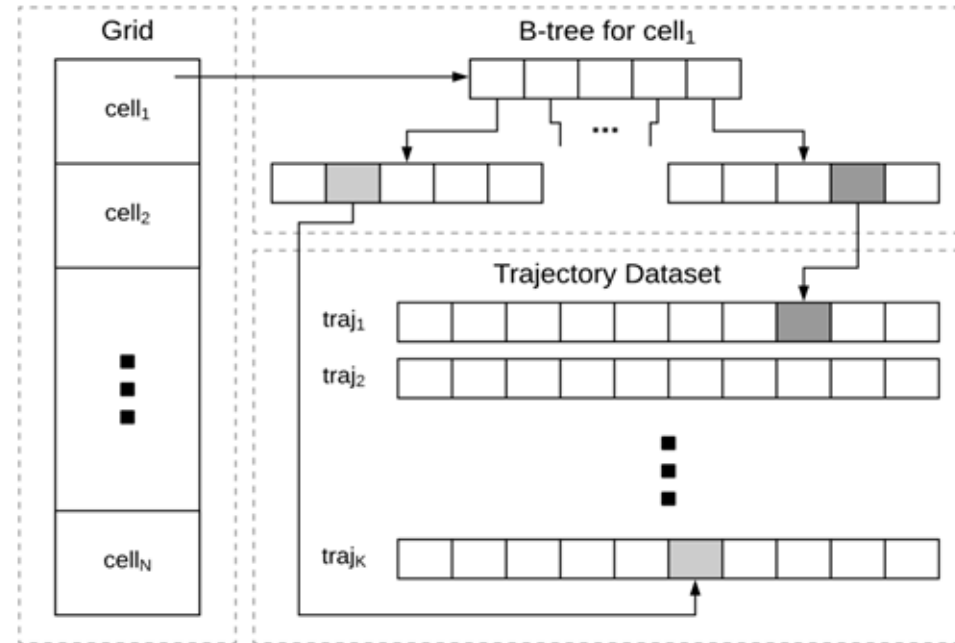


- **Routing** on *raw historical trajectories*
 - Only trajectories **spatially** and **temporarily** close to current position are considered.
- **Fallback** to the *road network* when trajectories are not available



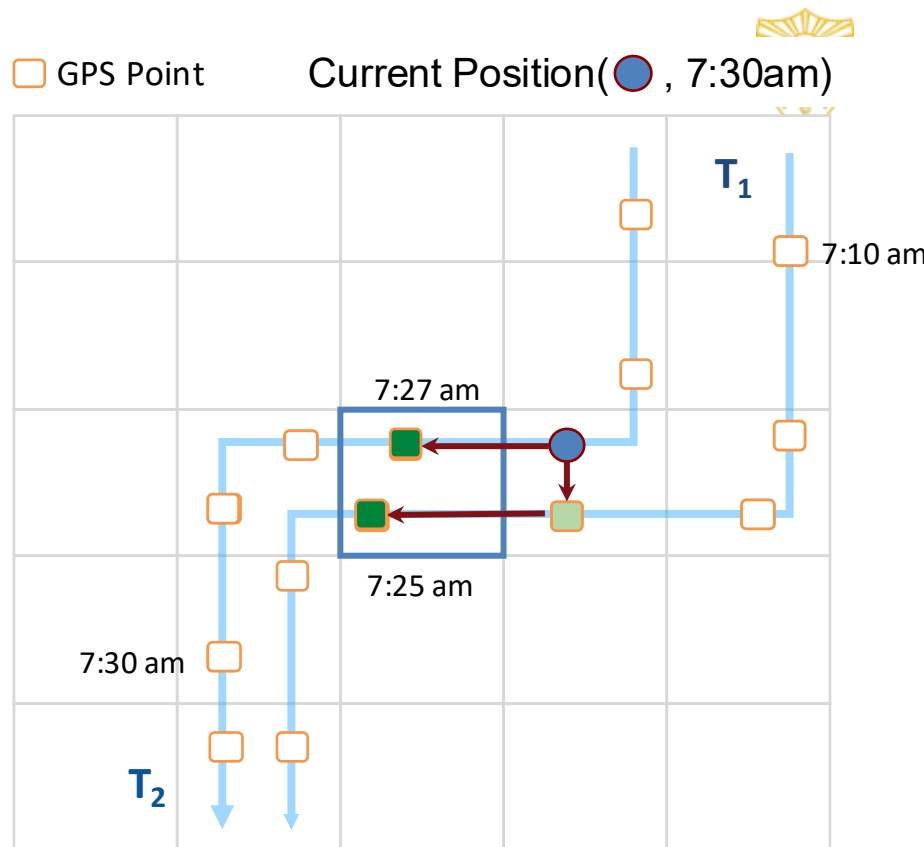
Index Structure

- Trajectory: temporal sequence $(\text{gps}_1, \text{gps}_2, \dots, \text{gps}_k)$ of tuples in the form of $\text{gps}_i = \langle \text{loc}, \text{ts} \rangle$
- *Each GPS measurement of the trajectories is assigned to the cell that it falls into*
- To efficiently retrieve only the measurements within a cell that satisfy the given departure (or arrival) time, a B-tree is constructed on the timestamps.



Neighbors Identification

- **Trajectory-based neighbor**
 - Next GPS point in the *same trajectory*
 - *Follows the natural path taken by the vehicle.*
- **Cross-trajectory neighbor**
 - Points from *other trajectories* whose previous point lies in the **same grid cell**
 - *Captures alternate paths that pass through the current location.*
- Only points recorded **close in time** (e.g., within ± 10 minutes) are examined.
 - *Selected neighbors are relevant to current traffic patterns.*



Neighbors Identification

- **Trajectory-based neighbor**

- Next GPS point in the *same trajectory*

- **Movement Cost :**

- $C_{traj} = ts(\text{green square}) - ts(\text{blue circle})$

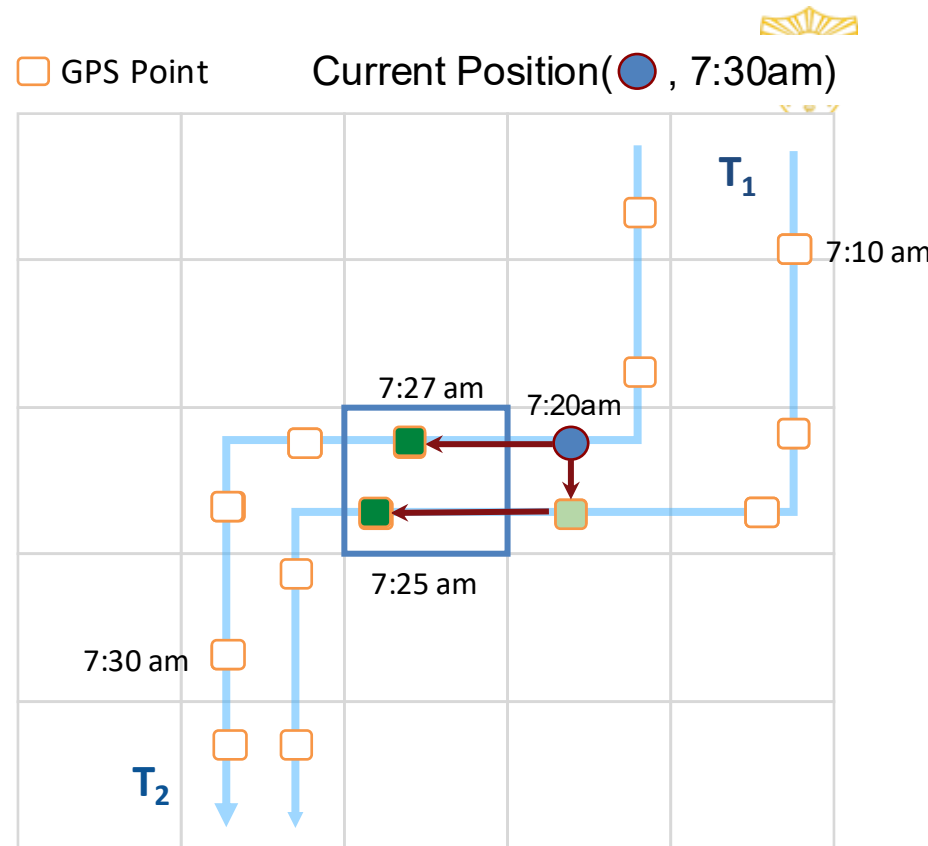
- **Cross-trajectory neighbor**

- Points from *other trajectories* whose previous point lies in the **same grid cell**

- **Movement Cost:**

- $C_{traj} = C_{trans} + [ts(\text{dark green square}) - ts(\text{light green square})]$

- C_{trans} : constant & depends on grid size



Query Point: $\langle p_{OR}, p_{DEST}, dtime \rangle$

[illegible]

TrajRoute: Approach

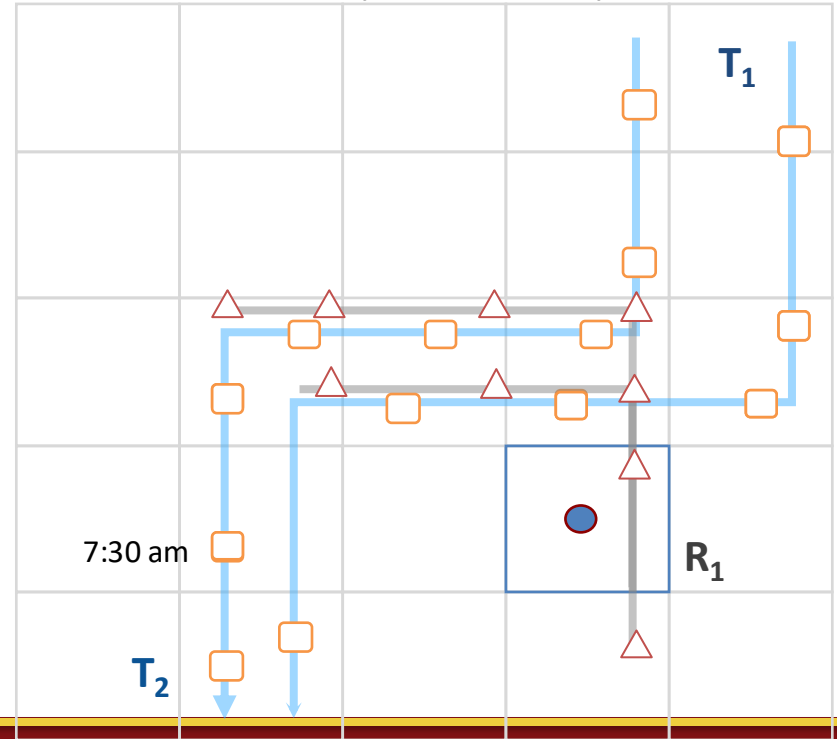


□ GPS Point: $\langle p_j = (\text{lat}_j, \text{lon}_j), \text{ts}_j \rangle$

△ Road Point: $\langle p_i = (\text{lat}_i, \text{lon}_i) \rangle$

Query Point: $\langle p_{\text{OR}}, p_{\text{DEST}}, \text{dtime} \rangle$

Current Position(● , 7:30am)



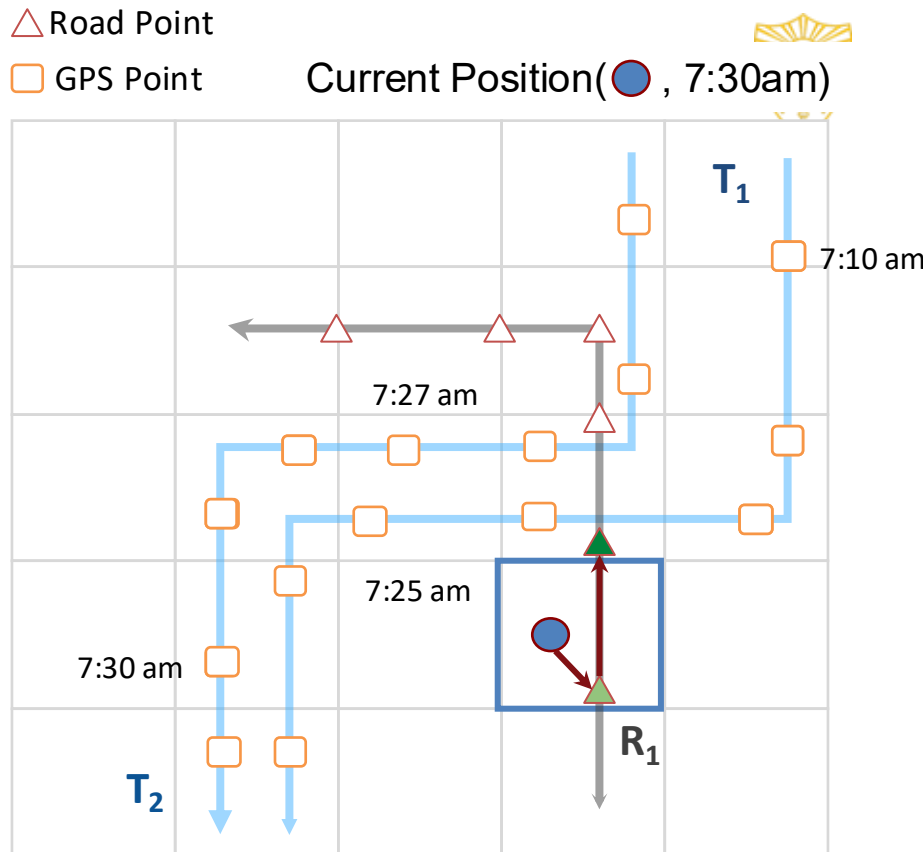
Fallback Mechanism

- **Motivation**

- Trajectories *may not fully cover* all areas or times.
- Road network *fills gaps* to ensure complete routing.
- **Note: Traditional approaches also lack traffic information for uncovered road segments**

- **Road neighbors (same notion):**

- Next point on the same road segment.
- Points from other segments in the **same cell**.



Fallback Mechanism

- **Road neighbors (same notion):**

- Next point on the same road segment.

- **Movement Cost :**

- $C_{road} = [\text{dist}(\triangle, \triangle) / v(\triangle)]$

- Points from other segments in the **same cell**.

- **Movement Cost :**

- $C_{road} = c_{trans} + [\text{dist}(\triangle, \triangle) / v(\triangle)]$

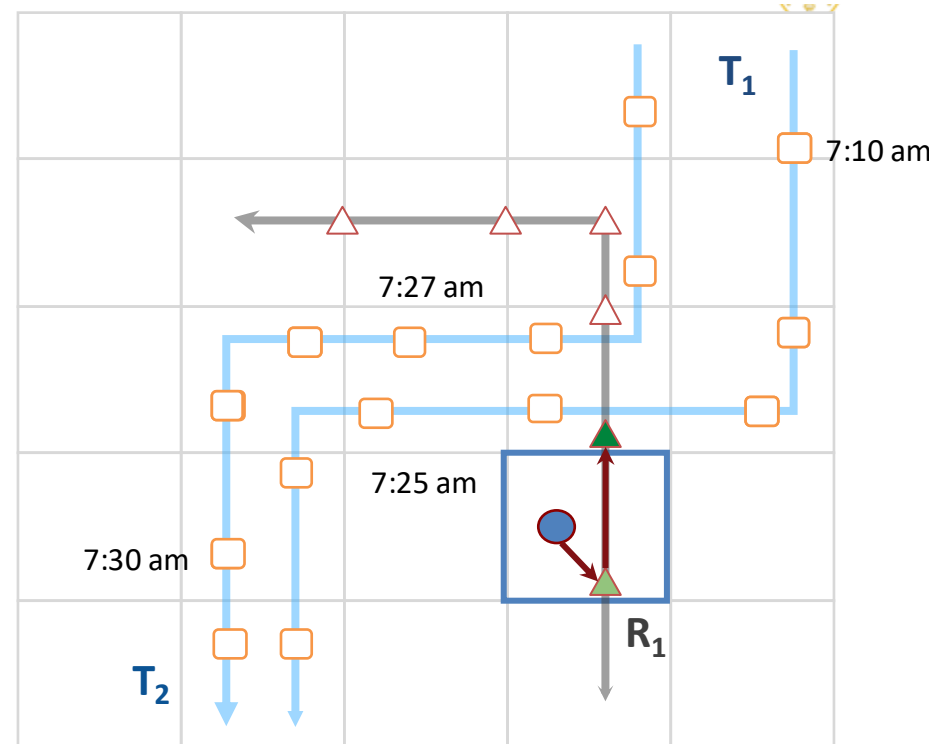
- *dist: Haversine distance*

- *v: road speed limit*

△ Road Point

□ GPS Point

Current Position(●, 7:30am)



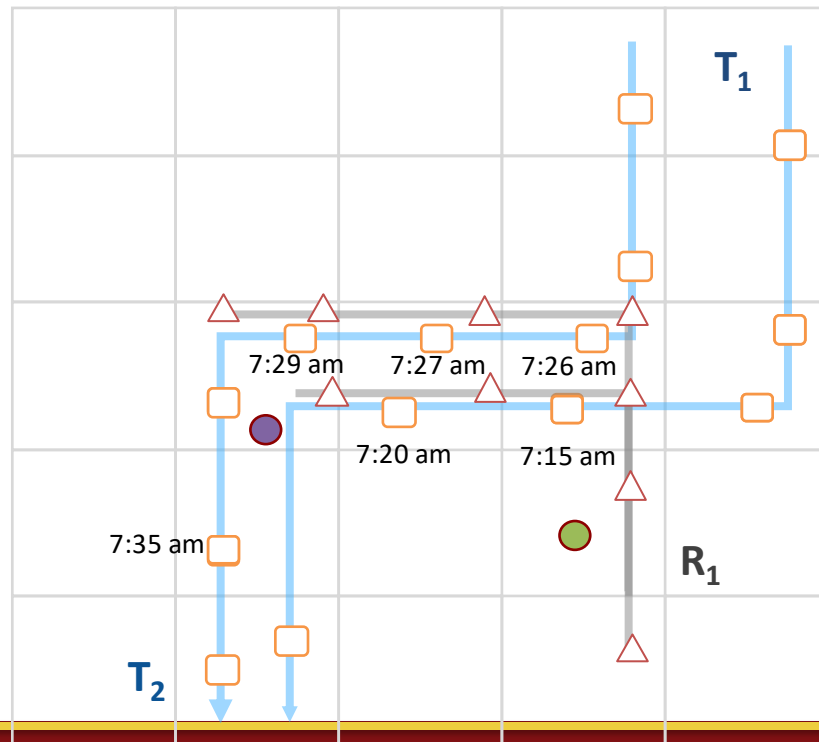
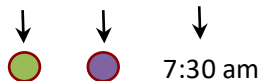
Example



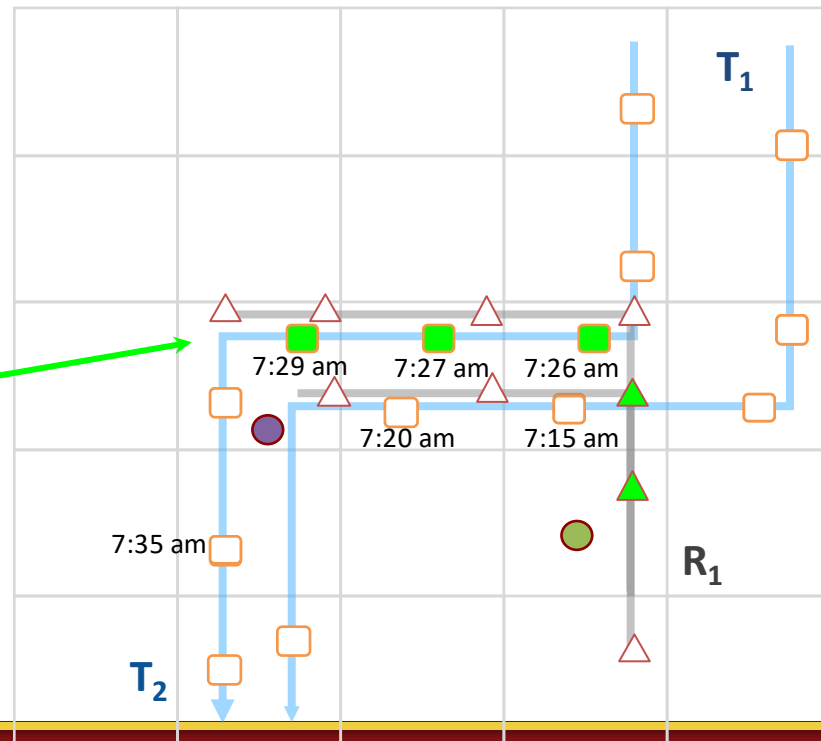
□ GPS Point: $\langle p_j = (\text{lat}_j, \text{lon}_j), \text{ts}_j \rangle$

△ Road Point: $\langle p_i = (\text{lat}_i, \text{lon}_i) \rangle$

Query Point: $\langle p_{\text{OR}}, p_{\text{DEST}}, \text{dtime} \rangle$



Ideal Path



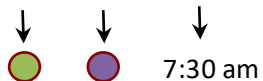
Example



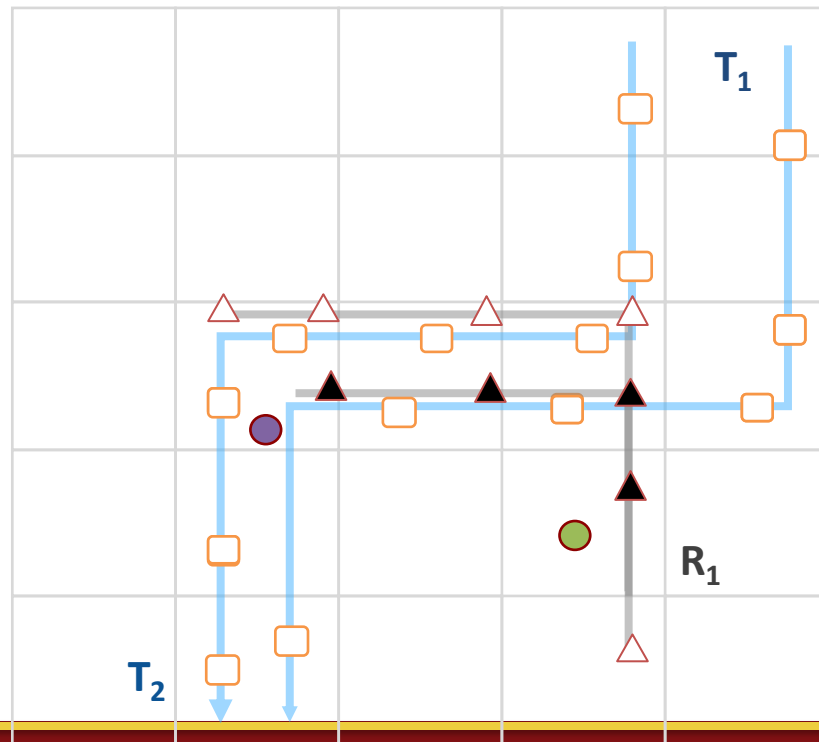
□ GPS Point: $\langle p_j = (\text{lat}_j, \text{lon}_j), \text{ts}_j \rangle$

△ Road Point: $\langle p_i = (\text{lat}_i, \text{lon}_i) \rangle$

Query Point: $\langle p_{\text{OR}}, p_{\text{DEST}}, \text{dtime} \rangle$



$C_{\text{road}} < C_{\text{traj}}$. Why?



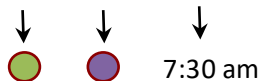
Example



□ GPS Point: $\langle p_j = (\text{lat}_j, \text{lon}_j), \text{ts}_j \rangle$

△ Road Point: $\langle p_i = (\text{lat}_i, \text{lon}_i) \rangle$

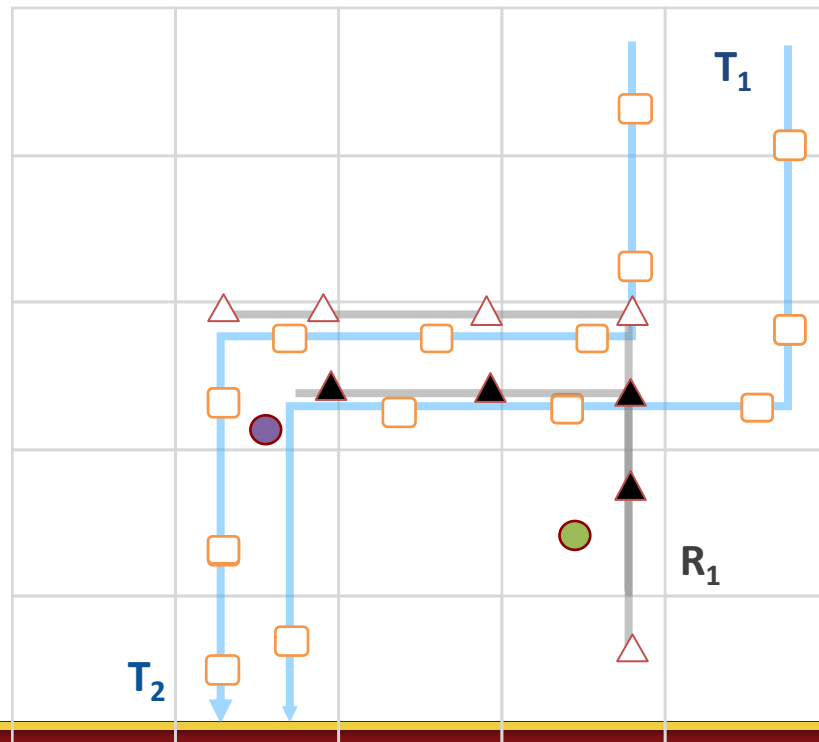
Query Point: $\langle p_{\text{OR}}, p_{\text{DEST}}, \text{dtime} \rangle$



Using speed limit does not account for:

- Intersection costs
- Acceleration/Deceleration
- Traffic Lights
- Traffic Congestion etc.

Inherently
encoded in
trajectory
timestamps



Fallback Mechanism

- **Issue!:**

- Road segment costs use **average speeds**, ignoring real-time traffic delays.
- **Trajectory costs are naturally higher**, reflecting real-world conditions.
- As a result, the system **favors road segments**, even when trajectories are more accurate.

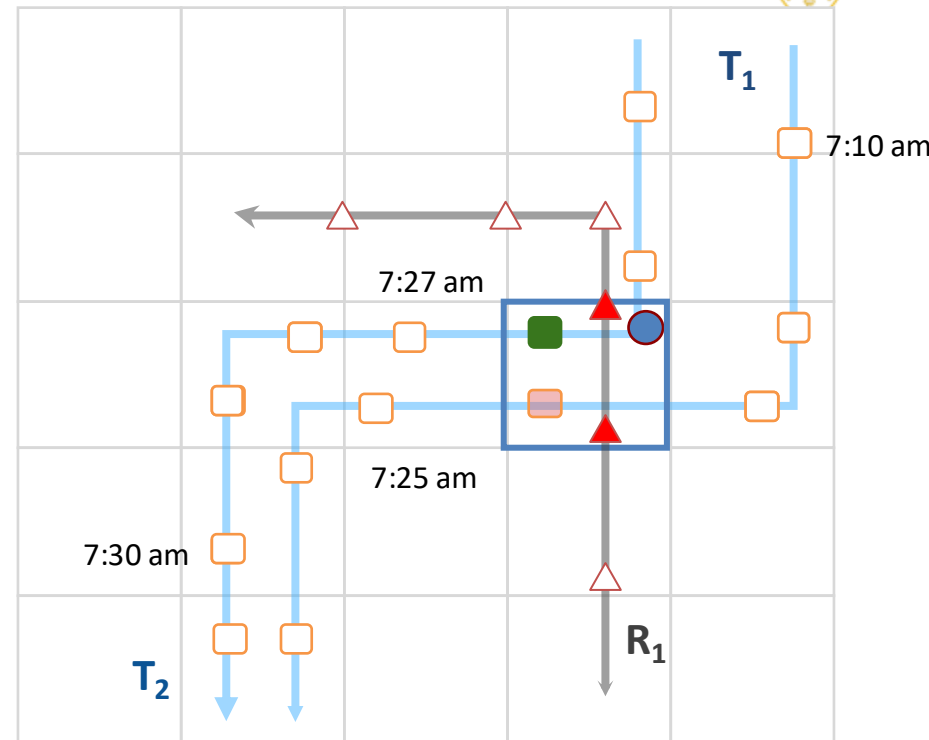
- **Cost Adjustment:**

- *Penalize roads:* $\text{Cost} = (1 + r_{\text{penalty}}) \cdot C_{\text{road}}$
- *Reward continuous trips:* $\text{Cost} = (e^{-r^w}) \cdot C_{\text{traj}}$

△ Road Point

□ GPS Point

Current Position(●, 7:30am)



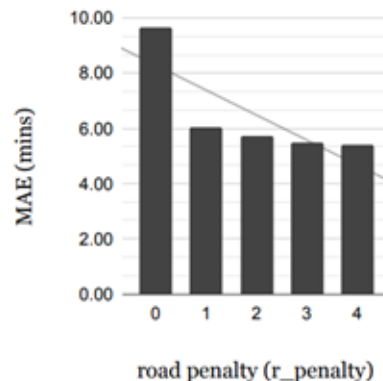
Origin (●, 7:10 am)

- $$h(p_k) = \frac{\text{dist}(p_k, Q.p_{DEST})}{v_{max}} \rightarrow \text{always underestimates the cost}$$

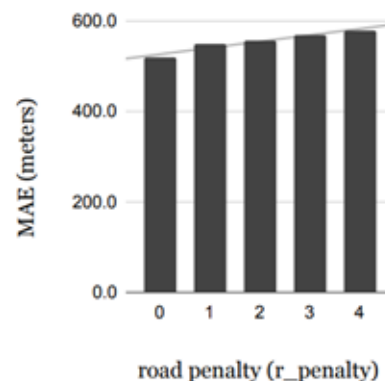


Experiments

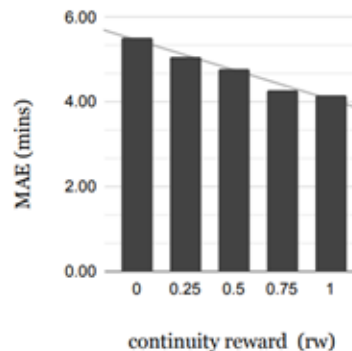
- **Data Source**
 - San Francisco Taxi Data
 - OSM for road network
- **Data Statistics**
 - > 1M trajectories, 27.279 roads
 - 99% spatial coverage
 - Peak: ~25%, Off-peak: ~75%
 - Weekend: ~35%, Weekday: ~65%
- **Evaluation**
 - *Random* Origin-Destination Queries from trajectories.
 - Comparison of routes with Azure Maps
 - Length of route
 - ETA of route
 - Experiment with different penalty ($r_penalty$) and reward values



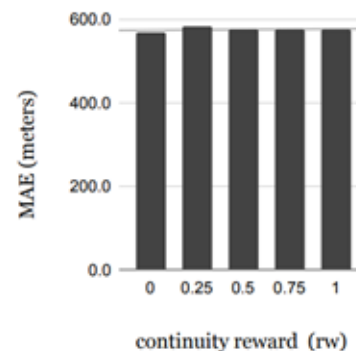
(a) MAE of route travel time



(b) MAE of route distance



(a) MAE of route travel time



(b) MAE of route distance

Experiments

• Evaluation

- Random Origin-Destination Queries from trajectories.
- Comparison of routes with Azure Maps
 - Length of route
 - ETA of route
- Spatial Coverage
 - Keep trajectories that cover $x\%$ of the area
 - Fixing $r_penalty=3.0$ and $rw=0.75$

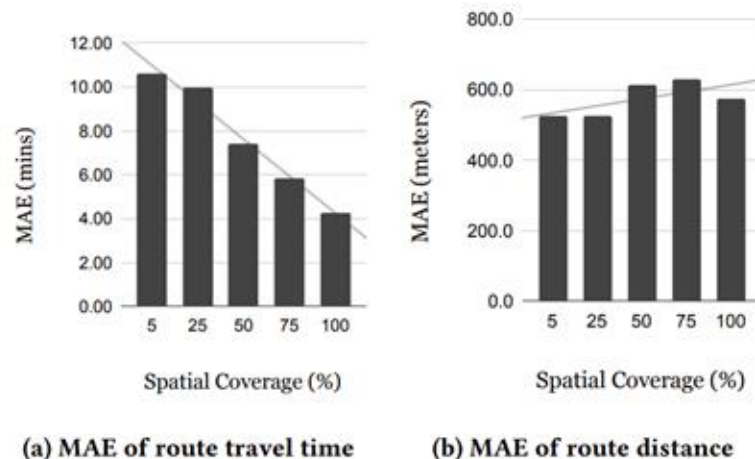
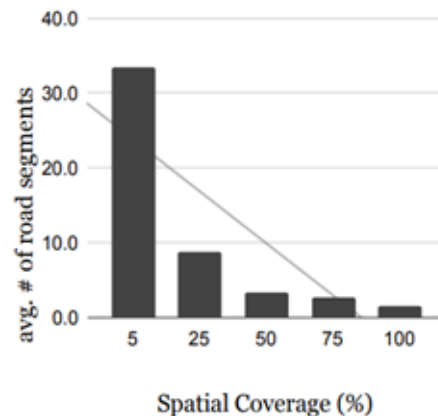


Figure 7: Results for different levels of spatial coverage.



Experiments



Query Time: 06:01 PM
Azure Maps ETA: 21 mins



(a) Route for $r_{penalty} = 0$ and $r_w = 0$.
TrajRoute ETA: 10 mins.



(b) Route for $r_{penalty} = 3$ and $r_w = 0$.
TrajRoute ETA: 16.21 mins.



(c) Route for $r_{penalty} = 3$ and $r_w = 0.75$.
TrajRoute ETA: 19.53 mins.

Query Time: 01:25 AM
Azure Maps ETA: 12 mins



(a) Route for $r_{penalty} = 0$ and $r_w = 0$.
TrajRoute ETA: 8.3 mins.



(b) Route for $r_{penalty} = 3$ and $r_w = 0$.
TrajRoute ETA: 10.15 mins.



(c) Route for $r_{penalty} = 3$ and $r_w = 0.75$.
TrajRoute ETA: 11.58 mins.

Conclusion

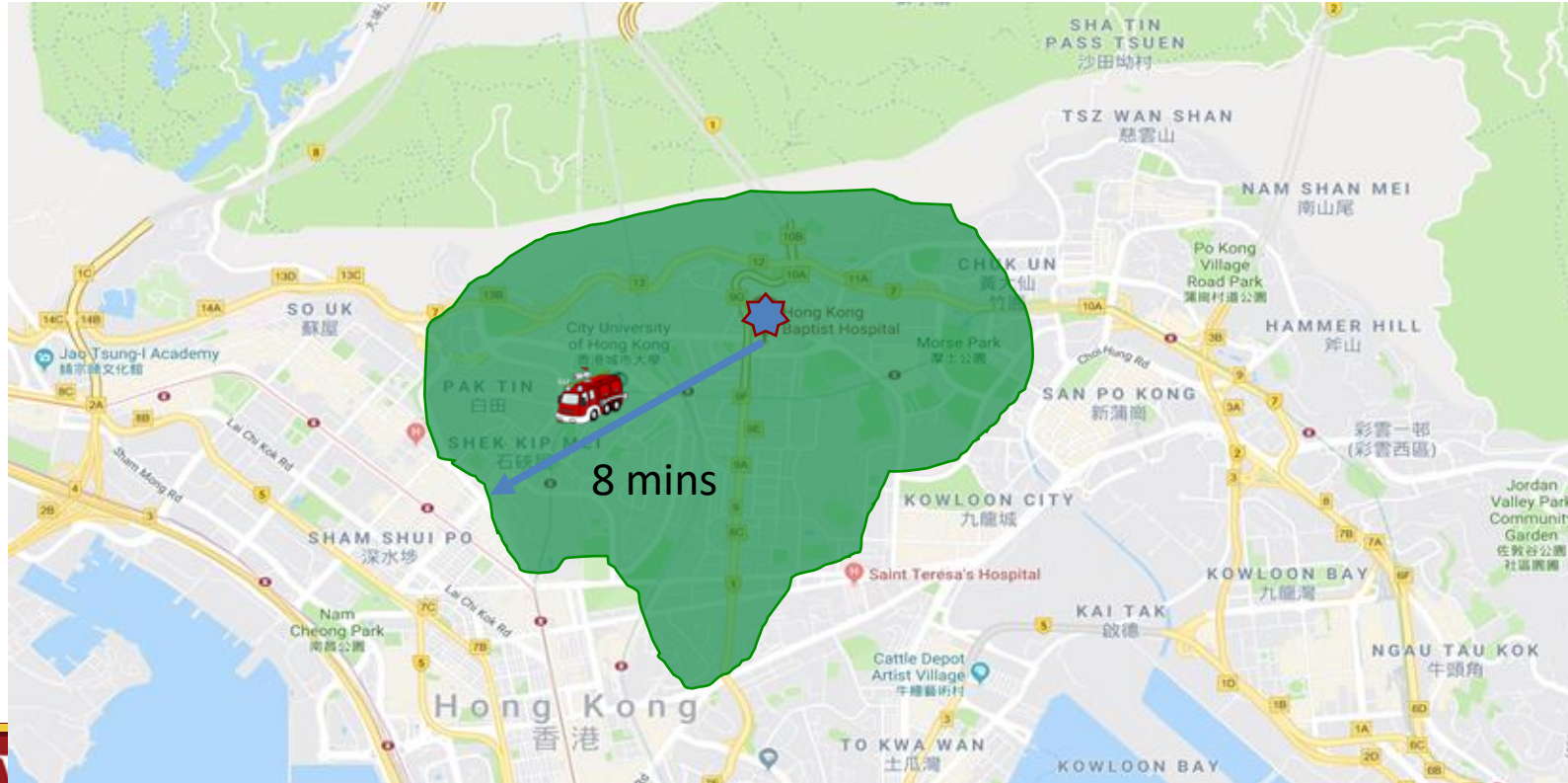


- Introduced **TrajRoute**, a routing algorithm that uses **raw trajectory data** directly.
 - *Reduces **infrastructure and maintenance costs** by avoiding heavy preprocessing steps!*
- Achieves **realistic** and **accurate routes**, with travel times and distances *comparable* to Azure Maps.
- **Captures real-world travel behavior** (e.g., traffic, stops, lights) implicitly through historical trajectories.
- **Intended for** dense, up-to-date trajectory data.
 - *Similar to existing systems that rely on historical traces.*
- **Future work:** Deploy TrajRoute in a **full navigation pipeline** for real-time use.

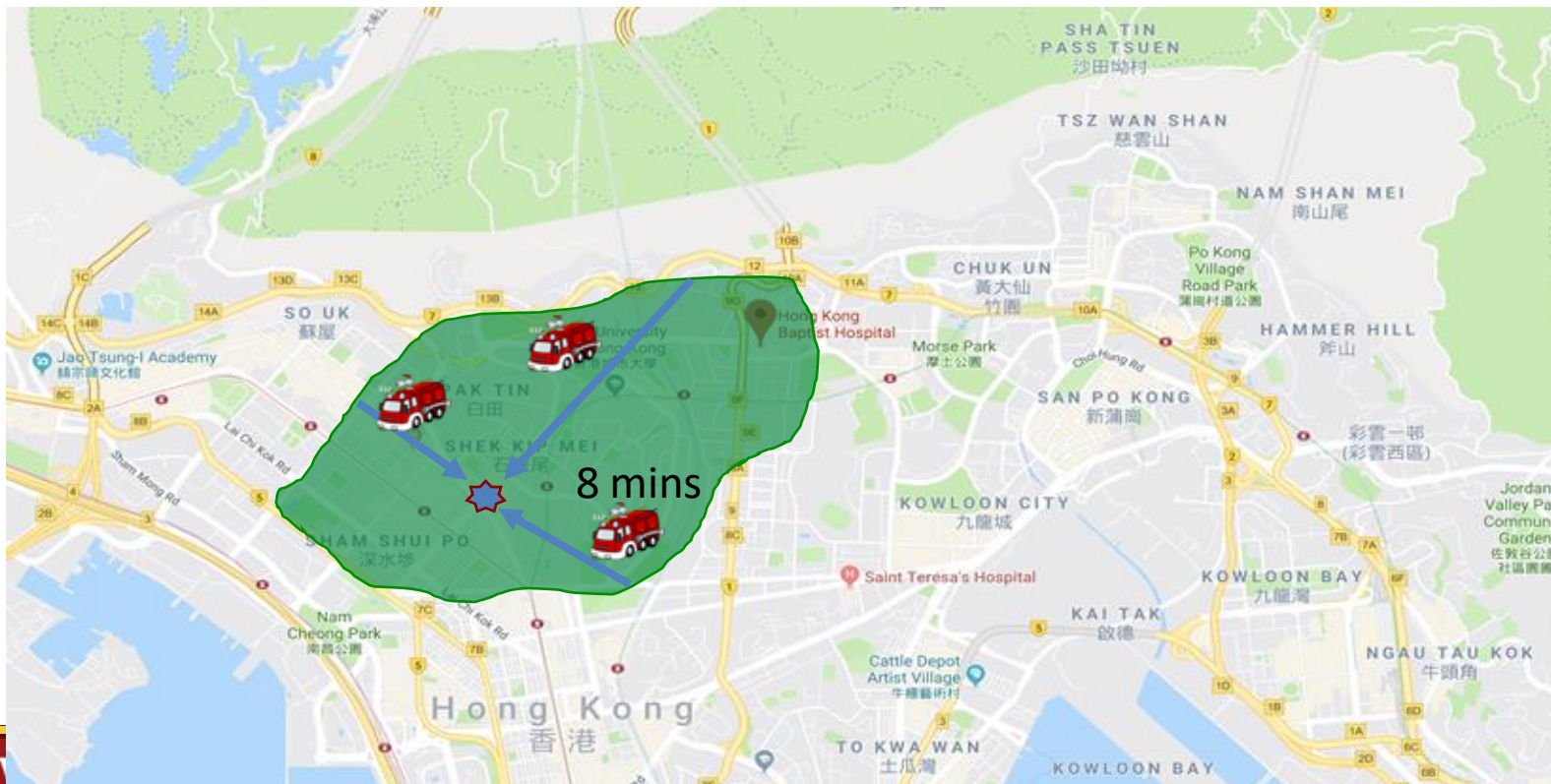
Isochrone Maps



**Actual travel times might vary*



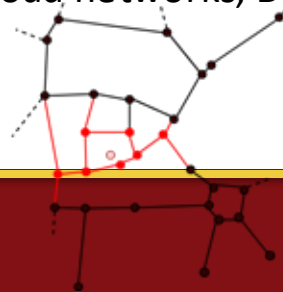
Reverse Reachability Analysis



Graph-based Approaches



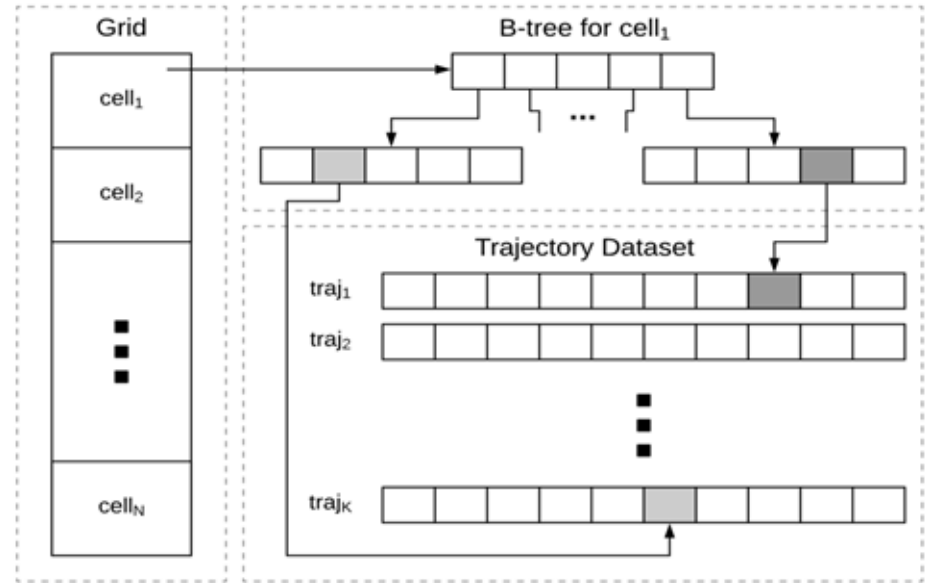
- **Isochrone maps** extensively studied in the databases community
- In **graph theory** defined as the minimal subgraph that can be reached from a query vertex given a limited path cost that is equivalent to travel time.
 - More precisely, it's the set of all reachable vertices, fully traversed edges, and possibly partially traversed edges
- Standard solutions are based on **Dijkstra's** (or **Dreyfus**) shortest path algorithm
 - **[ICDE'06]** Finding fastest paths on a road network with speed patterns, *Kanoulas et al.*
 - **[GIS'08]** Computing isochrones in multi-modal, schedule-based transport networks, *Bauer et al.*
 - **[EDBT'08]** Finding time-dependent shortest paths over large graphs, *Ding et al.*
 - **[CIKM'11]** Defining isochrones in multi-modal spatial networks, *Gamber et al.*
 - **[SEA'16]** Fast exact computation of isochrones in road networks, *Baum et al.*



Data-Driven Reachability



- Remove the *expensive map-matching* step
 - Can take days to compute time-dependent weights for big data
- Remove the traversal step of *complex graphs*
 - The higher the query time limit the more edges need to be explored
- Compute isochrone maps *directly from data*
 - Only process trajectories that satisfy query criteria
- Support multiple *Reachability Queries*
 - Single-Source & Multi-Source (Normal)
 - Single-Target & Multi-Target (Reverse)



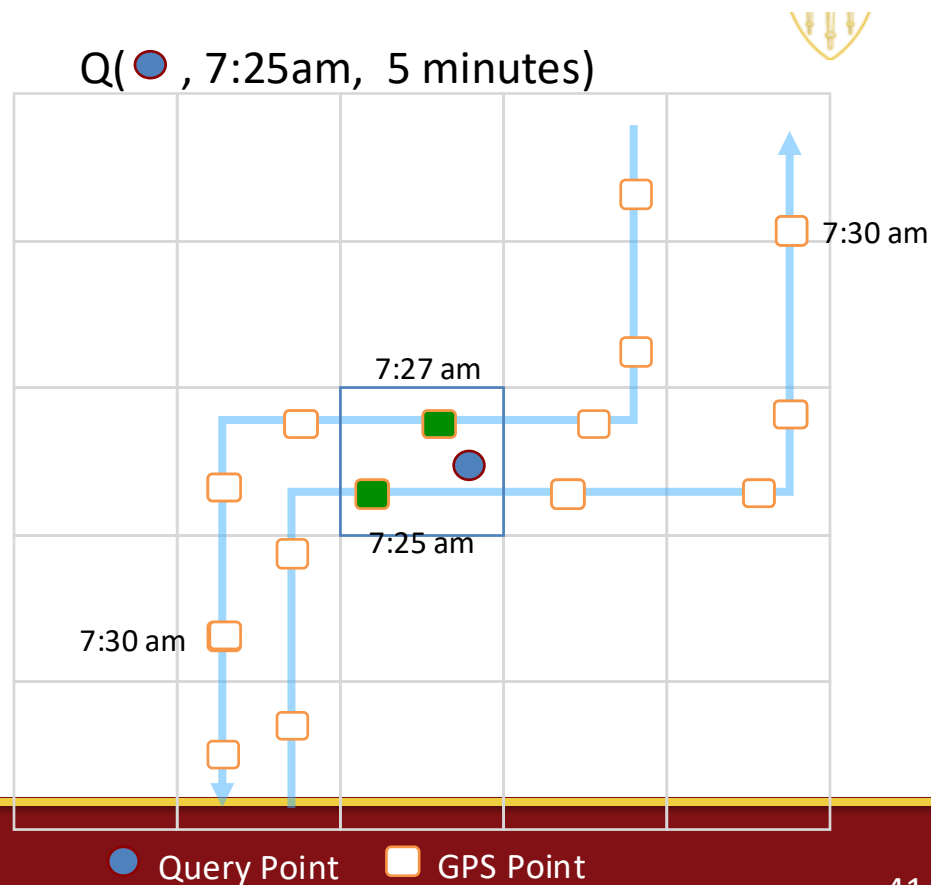
Single-Source & Multi-Source Queries

Reachability Query

- $Q(s, t, d)$

- s : source location
- t : departure time
- d : time limit in minutes

```
1:  $c \leftarrow findCell(G, Q.s)$ 
2:  $r \leftarrow \{\}$  // Initialize result to empty set
3: for  $(traj, i) \in c.gpsInWindow(Q.t, Q.t + Q.d)$  do
4:   while  $i < traj.length$  and  $traj[i].ts \leq Q.t + Q.d$  do
5:      $r \leftarrow r \cup \{traj[i].loc\}$ 
6:      $i \leftarrow i + 1$ 
7:   end while
8: end for
9: return  $r$  // Return the set of all reachable points
```



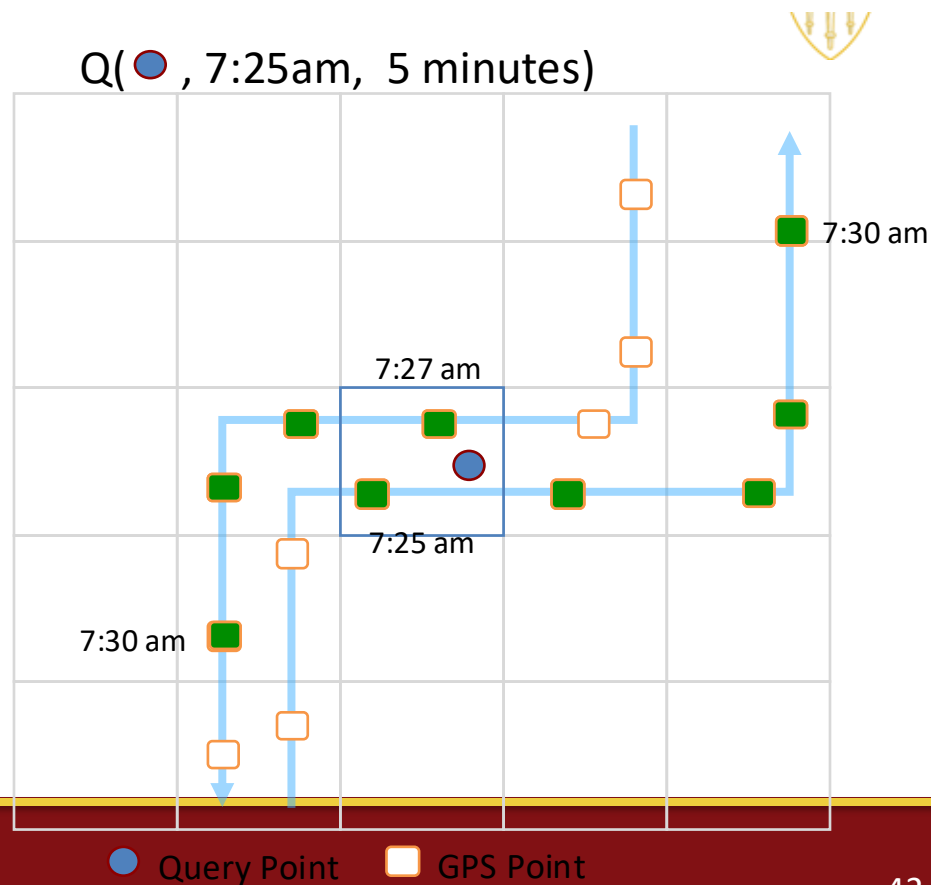
Single-Source & Multi-Source Queries

Reachability Query

- $Q(s, t, d)$

- s : source location
- t : departure time
- d : time limit in minutes

```
1:  $c \leftarrow findCell(G, Q.s)$ 
2:  $r \leftarrow \{\}$  // Initialize result to empty set
3: for  $(traj, i) \in c.gpsInWindow(Q.t, Q.t + Q.d)$  do
4:   while  $i < traj.length$  and  $traj[i].ts \leq Q.t + Q.d$  do
5:      $r \leftarrow r \cup \{traj[i].loc\}$ 
6:      $i \leftarrow i + 1$ 
7:   end while
8: end for
9: return  $r$  // Return the set of all reachable points
```



Single-Target & Multi-Target Queries

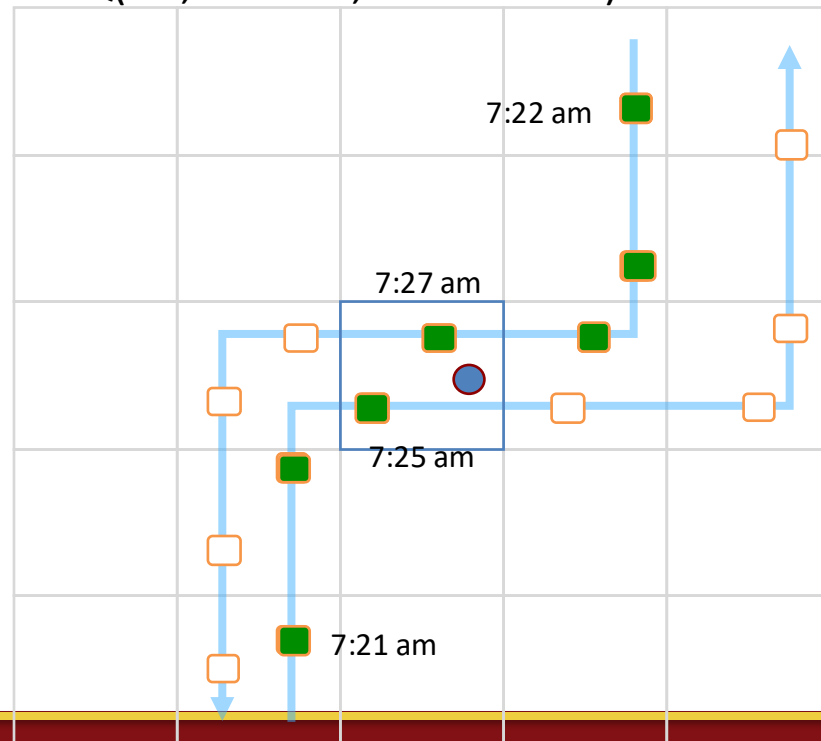
Reverse Reachability Query

- $Q(q, t, d)$

- q : target location
- t : arrival time
- d : time limit in minutes

```
1:  $c \leftarrow \text{findCell}(G, Q.q)$ 
2:  $r \leftarrow \{\}$  // Initialize result to empty set
3: for  $(\text{traj}, i) \in c.\text{gpsInWindow}(Q.t - Q.d, Q.t)$  do
4:   while  $i \geq 0$  and  $Q.t - Q.d \leq \text{traj}[i].ts$  do
5:      $r \leftarrow r \cup \{\text{traj}[i].loc\}$ 
6:      $i \leftarrow i - 1$ 
7:   end while
8: end for
9: return  $r$  // Return the set of all reachable points
```

$Q(\bullet, 7:30\text{am}, 10 \text{ minutes})$

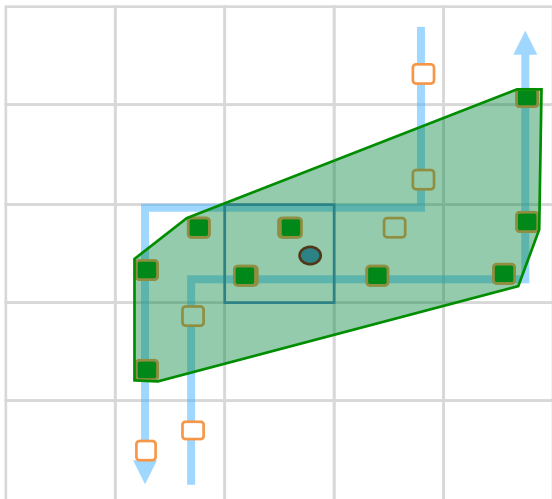


● Query Point □ GPS Point

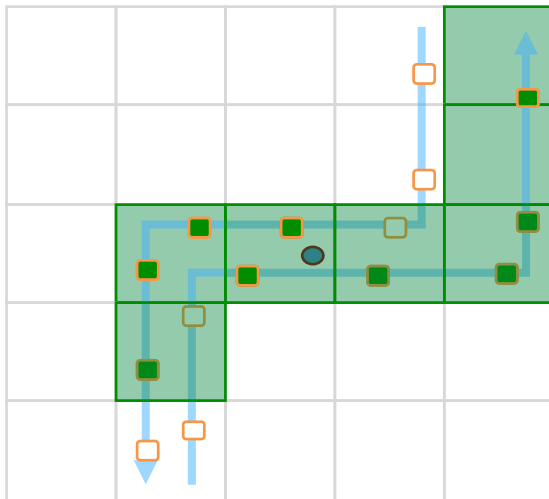
Visualization Methods



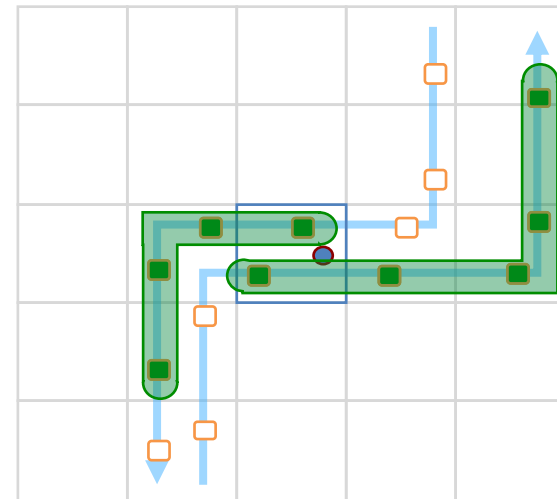
Convex Hull



Cells



Trajectory Buffer



● Query Point □ GPS Point

Visualization Methods



(a) Convex Hull



(b) Cells



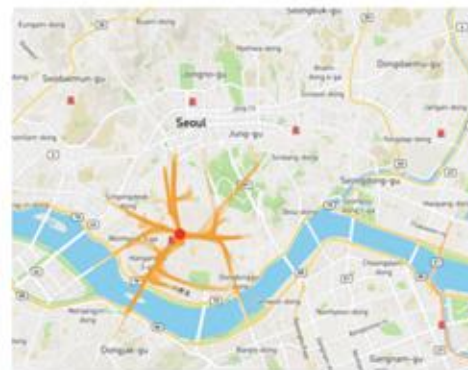
(c) Buffers



(d) Convex Hull Reverse



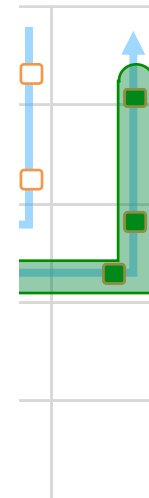
(e) Cells Reverse



(f) Buffers Reverse

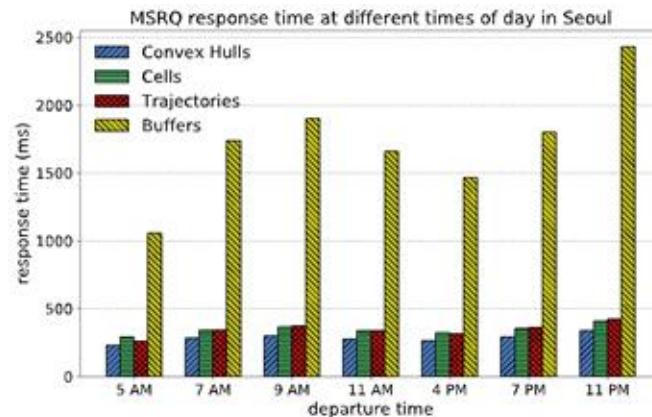


er



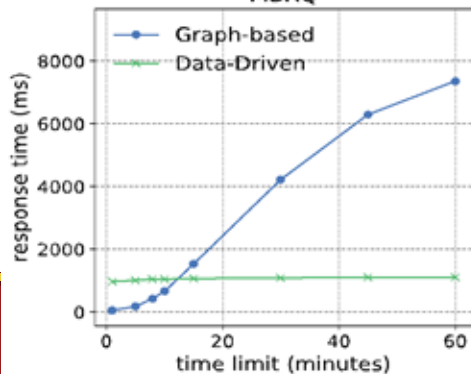
Experiments

- Data Source
 - Navicall (Seoul Brand Taxi Call Company)
- Data collection period
 - July 2016 – November 2016
- Data Statistics
 - 5,000 taxies
 - 1 min unit sensing data
 - ~600M readings
 - ~50 GB total

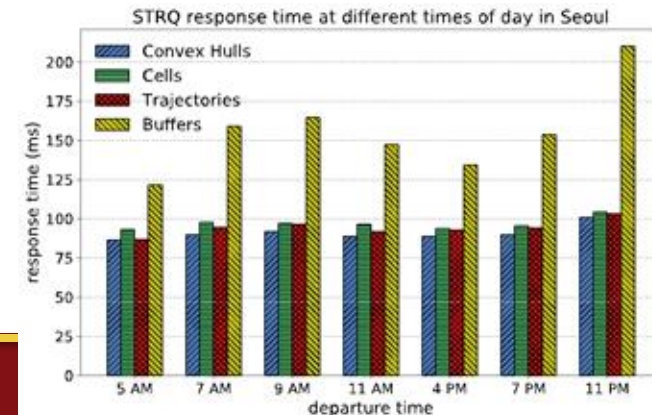
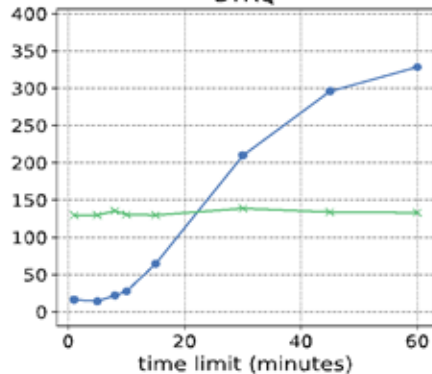


Graph-Based vs Data-Driven Query Processing

MSRQ



STRQ



References



- [1] Newson P. & Krumm J., “Hidden Markov map matching through noise and sparseness,” Proceedings of the 17th ACM SIGSPATIAL International Conference on Advances in Geographic Information Systems GIS 09, 336, 2009
- [2] [DiDi’s IJCAI-19 Tutorial: Artificial Intelligence in Transportation](#) (slides 28–40)
- [3] [Map Matching @ Uber](#)
- [4] Anastasiou, C., Huang, C., Kim, S. H., & Shahabi, C. (2019, June). Time-Dependent Reachability Analysis: A Data-Driven Approach. In *2019 20th IEEE International Conference on Mobile Data Management (MDM)* (pp. 138-143). IEEE.
- [5] Siampou, MD., Anastasiou, C., Krumm, J., & Shahabi, C. (2024). TrajRoute: Rethinking Routing with a Simple Trajectory-based Approach: Forget the Maps and Traffic!. To *appear @ IEEE International Conference on Mobile Data Management (MDM)* .